

PRODUCT ARCHITECTURE PLAN

Velvety Dawn

MailFlow

AI-Powered Multi-Account Email Outreach SaaS
Architecture & Implementation Plan

Prepared 2026-05-26 · v1.0

Table of Contents

Table of Contents

1

Velvety Dawn — MailFlow SaaS Plan

2

1. Context

2

2. Locked-in Tech Stack

2

3. Feature Inventory

3

4. Monorepo Folder Structure

5

5. Database Schema (MongoDB / Mongoose)

6

6. API Design

8

7. Queue & Worker Design (BullMQ)

9

8. Auth.js v5 Flow

11

9. AI Automation Flow (OpenRouter)

11

10. Email Processing Pipeline (end to end)

12

11. Dashboard UI Structure

12

12. Recommended Packages

13

13. Step-by-Step Implementation Roadmap

14

14. Production Deployment Strategy

15

15. Security Best Practices

16

16. Critical Files (when implementation starts)

16

17. Verification (end to end)

17

18. Deliverable Format

18

Velvety Dawn — MailFlow SaaS Plan

*AI-powered, multi-account email outreach & automation platform. Working dir: `c:\digital`
`guru\webApp\mailflow` (greenfield).*

1. Context

Why we're building this: Operators running cold-outreach and promotional email campaigns juggle (a) many sending mailboxes to dodge per-account rate limits, (b) lead lists in spreadsheets, (c) replies scattered across inboxes, and (d) manual follow-ups that go stale. MailFlow consolidates account rotation, queue-based sending, AI-powered reply intent detection, automated follow-ups, a unified inbox, and a CRM into one production-grade SaaS.

Intended outcome: A multi-tenant SaaS where an admin connects N Gmail/SMTP accounts, uploads leads, launches a campaign, and the platform autonomously sends, tracks replies, classifies intent with AI, follows up via workflow rules, and grants lifetime premium access when a configured promotion condition is met — all observable from a unified inbox + analytics dashboard.

Approach: MVP-first vertical slice (auth → connect 1 account → send 1 campaign → see replies in unified inbox → AI classify → auto-follow-up), then horizontal scaling (multi-account rotation, workflow builder, analytics, reward automation). Modular monorepo so the worker fleet can scale independently from the Next.js app.

2. Locked-in Tech Stack

Layer	Choice
Framework	Next.js 15 (App Router) + React 19 + TypeScript
UI	TailwindCSS + shadcn/ui + lucide-react + Radix primitives
Auth	Auth.js v5 (NextAuth) — credentials + Google OAuth (also used for Gmail account connection consent flow)
DB	MongoDB (Atlas) via Mongoose
Cache + Queue	Redis (Upstash for serverless / Railway for workers) + BullMQ
AI	OpenRouter (single key, route to Claude Sonnet/Haiku, GPT-4o-mini, Gemini etc. per task)
Email send	Gmail API (OAuth per connected account), Nodemailer SMTP, optional SendGrid/Mailgun providers

Layer	Choice
Email receive	Gmail API <code>users.watch</code> + Pub/Sub push, SMTP IMAP poll fallback
Background jobs	BullMQ workers (separate Node process)
Validation	Zod (shared schemas in <code>packages/shared</code>)
Observability	Pino logs, Sentry, BullBoard for queue inspection
Deploy	Vercel (Next.js app) + Railway (workers + Redis + Mongo proxy)

3. Feature Inventory

Core modules

1. **Auth & RBAC** — email/password + Google OAuth, roles: `superadmin`, `admin`, `member`, `viewer`. Per-org isolation.
2. **Multi-email account manager** — connect Gmail (OAuth), SMTP (creds), provider APIs (SendGrid/Mailgun). Health checks, daily-cap tracking, warmup-ready.
3. **Lead/Contact CRM** — collections of `Contact` per org, CSV import/export, dedupe, tags, lists, custom fields, GDPR unsubscribe.
4. **Campaign builder** — wizard: pick list → pick sender pool → pick template → schedule → send-rate config.
5. **Template engine** — Handlebars-style merge tags (`{{firstName}}`), per-variant A/B testing, snippet library.
6. **Send orchestrator** — BullMQ jobs with sender rotation, per-account hourly/daily cap, jittered delays, retry with backoff.
7. **Inbound email processor** — Gmail watch + IMAP poll, thread reconciliation, normalize into `Message` documents.
8. **AI reply analyzer** — classify intent (interested / not interested / question / unsubscribe / OOO / bounce / spam), extract entities, generate suggested reply.
9. **Workflow automation engine** — trigger (reply received, no-reply after N days, link click, intent=X) → conditions → actions (send template, change list, notify, grant reward, pause).
10. **Unified inbox** — Gmail-like view across all connected mailboxes per org, with AI summary, suggested reply, and one-click send.
11. **Reward / lifetime-premium automation** — admin defines promo conditions (e.g., "replied positive + clicked link + filled form"); trigger fires → grants lifetime premium on configured target system via webhook/API.

12. **Analytics dashboard** — sent/delivered/bounced/opened/clicked/replied, per-campaign funnel, per-sender health, AI intent breakdown.

13. **Admin settings** — org settings, billing (future), API keys, webhook endpoints, audit log.

Supporting features

- Email scheduling (cron-like + per-recipient timezone send)
- Sender rotation strategies: round-robin, weighted, least-recently-used
- Retry mechanism with exponential backoff + dead-letter queue
- Rate limiting at queue + per-account level (token-bucket in Redis)
- Email thread tracking via `Message-ID` / `In-Reply-To` / `References` headers
- AI-generated replies (one-click accept / edit)
- Workflow triggers (reply, no-reply, time-elapsed, intent, manual)
- Webhook handling (Gmail Pub/Sub push, inbound provider webhooks, outbound webhooks for reward grant)
- Campaign templates library + duplication
- CSV import with column mapping + dry-run validation
- Spam prevention: per-account warmup ramp, link sanitation, image hosting, plain-text alt
- SPF/DKIM/DMARC guidance UI per connected domain
- Activity logs (immutable audit trail per org)

4. Monorepo Folder Structure

```

mailflow/
├── apps/
│   ├── web/                                # Next.js 15 (App Router) – UI + API routes (BFF)
│   │   ├── app/
│   │   │   ├── (auth)/                    # signin, signup
│   │   │   ├── (dashboard)/
│   │   │   ├── inbox/                    # unified inbox
│   │   │   ├── campaigns/
│   │   │   ├── contacts/
│   │   │   ├── workflows/
│   │   │   ├── accounts/                # connected email accounts
│   │   │   ├── analytics/
│   │   │   ├── settings/
│   │   │   └── api/
│   │   │       ├── auth/[...nextauth]/
│   │   │       ├── webhooks/gmail/
│   │   │       ├── webhooks/sendgrid/
│   │   │       ├── trpc/                # or REST handlers
│   │   │       └── layout.tsx
│   │   ├── components/                  # shadcn/ui components + app-specific
│   │   ├── lib/
│   │   └── middleware.ts                # auth gate
│   └── worker/                          # Standalone Node process – BullMQ workers
│       ├── src/
│       │   ├── workers/
│       │   │   ├── send-email.worker.ts
│       │   │   ├── inbound-email.worker.ts
│       │   │   ├── ai-analyze.worker.ts
│       │   │   ├── workflow-run.worker.ts
│       │   │   └── reward-grant.worker.ts
│       │   ├── schedulers/              # repeating jobs (gmail watch refresh, IMAP poll)
│       │   └── index.ts
│       └── package.json
├── packages/
│   ├── shared/                          # zod schemas, types, constants
│   ├── db/                             # mongoose models + connection helper
│   │   ├── models/
│   │   │   ├── User.ts
│   │   │   ├── Org.ts
│   │   │   ├── EmailAccount.ts
│   │   │   ├── Contact.ts
│   │   │   ├── List.ts
│   │   │   ├── Campaign.ts
│   │   │   ├── CampaignRecipient.ts
│   │   │   ├── Template.ts
│   │   │   ├── Message.ts
│   │   │   ├── Thread.ts
│   │   │   ├── Workflow.ts
│   │   │   ├── WorkflowRun.ts
│   │   │   ├── Reward.ts
│   │   │   ├── AuditLog.ts
│   │   │   └── ApiKey.ts
│   ├── ai/                             # OpenRouter client + prompt templates
│   └── client.ts

```

```

■ ■ ■■ prompts/
■ ■ ■ ■■ intent-classifier.ts
■ ■ ■ ■■ reply-generator.ts
■ ■ ■ ■■ summarizer.ts
■ ■ ■■ router.ts           # pick model per task (Haiku for classify, Sonnet for draft)
■ ■■ email/               # gmail/smtp/sendgrid adapters behind one interface
■ ■ ■■ providers/
■ ■ ■ ■■ gmail.ts
■ ■ ■ ■■ smtp.ts
■ ■ ■ ■■ sendgrid.ts
■ ■ ■■ sender.ts          # SenderRotator + RateLimiter
■ ■ ■■ parser.ts          # mailparser for inbound
■ ■■ queue/               # BullMQ queues + job types
■ ■ ■■ queues.ts
■ ■ ■■ jobs.ts
■ ■ ■■ rate-limiter.ts    # token-bucket per account
■ ■■ workflows/          # engine: triggers → conditions → actions
■ ■ ■■ engine.ts
■ ■ ■■ triggers/
■ ■ ■■ actions/
■ ■ ■■ dsl.ts
■ ■■ ui/                  # shadcn/ui re-exports + shared components
■■■ .env.example
■■■ docker-compose.yml    # local mongo + redis
■■■ turbo.json            # Turborepo
■■■ pnpm-workspace.yaml
■■■ package.json

```

5. Database Schema (MongoDB / Mongoose)

Every doc has `orgId` for multi-tenant isolation; indexed on every collection.

User

```
_id, orgId, email (unique), name, image, role: enum, hashedPassword?, emailVerified, createdAt
```

Org

```
_id, name, slug (unique), ownerId, plan, settings: { unsubscribeFooter, brandColor, ... }, createdAt
```

EmailAccount (the connected sending mailbox)

```

_id, orgId, provider: 'gmail'|'smtp'|'sendgrid'|'mailgun',
displayName, fromEmail, fromName,
auth: { accessToken (encrypted), refreshToken (encrypted), expiresAt } // gmail
      | { host, port, user, pass (encrypted), secure }                 // smtp
      | { apiKey (encrypted), domain }                                 // sendgrid/mailgun
limits: { dailyCap, hourlyCap, warmupDay },
health: { status, lastError, lastSentAt, sentToday, bouncesToday },
watchExpiration: Date,                                               // gmail push
historyId: String,                                                  // gmail watermark
createdAt

```

Contact

```
_id, orgId, email (compound-unique with orgId), firstName, lastName,
customFields: Map, tags: [String], listIds: [ObjectId],
status: 'active'|'bounced'|'unsubscribed'|'complained',
source, createdAt
```

List — _id, orgId, name, contactCount, createdAt

Template

```
_id, orgId, name, subject, bodyHtml, bodyText, mergeTags: [String],
variants: [{ name, subject, bodyHtml }], category, createdAt
```

Campaign

```
_id, orgId, name, status: 'draft'|'scheduled'|'running'|'paused'|'completed',
listIds: [ObjectId], senderPoolIds: [ObjectId],
templateId, schedule: { startAt, sendWindow: {tz, hours}, perRecipientCap },
rotation: 'round-robin'|'weighted'|'lru',
stats: { queued, sent, delivered, bounced, opened, clicked, replied, unsubscribed },
createdAt, createdBy
```

CampaignRecipient (one per send attempt)

```
_id, orgId, campaignId, contactId, emailAccountId,
status: 'queued'|'sending'|'sent'|'failed'|'bounced',
messageId, threadId, scheduledFor, sentAt, attempts, lastError,
events: [{ type:'open'|'click'|'bounce'|'reply'|'unsubscribe', at, meta }]
```

Thread

```
_id, orgId, emailAccountId, subject, participants: [email],
campaignId?, contactId?, lastMessageAt, messageCount,
aiSummary, aiIntent, status: 'open'|'closed'
```

Message (any inbound or outbound mail)

```
_id, orgId, threadId, emailAccountId, direction: 'in'|'out',
messageId (rfc822), inReplyTo, references: [String],
from, to: [String], subject, snippet, bodyHtml, bodyText, attachments: [...],
receivedAt|sentAt, ai: { intent, confidence, entities, draftReply }
```

Workflow

```
_id, orgId, name, enabled,
trigger: { type, params }, // e.g. {type:'reply_received', filters:{intent:'interested'}}
steps: [ { type:'condition'|'action', config } ], // DSL
createdAt
```


WorkflowRun — `_id`, `workflowId`, `orgId`, `contextRef`, `status`,
`log:[...]`, `startedAt`, `finishedAt`

Reward

```
_id, orgId, name, condition: { matchAll:[...] },
grantAction: { type:'webhook'|'api', url, payloadTemplate, secret },
recipients: [{ contactId, grantedAt, ref }]
```

AuditLog — `_id`, `orgId`, `actorId`, `action`, `target`, `meta`, `at` (TTL or cold storage after 90d)

ApiKey — `_id`, `orgId`, `name`, `hashedKey`, `scopes:[String]`, `createdBy`,
`lastUsedAt`

Indexes (critical)

- **Contact**: {`orgId:1`, `email:1`} unique
- **Message**: {`orgId:1`, `threadId:1`}, {`messageId:1`} unique
- **CampaignRecipient**: {`campaignId:1`, `status:1`}, {`scheduledFor:1`, `status:1`}
- **EmailAccount**: {`orgId:1`, `provider:1`}, {`watchExpiration:1`} (for refresh scheduler)

6. API Design

REST under `apps/web/app/api/` (optional tRPC layer for typed internal calls). All non-webhook routes require `Auth.js` session + `org` middleware.

```
POST  /api/auth/[...nextauth]           # Auth.js
GET   /api/me

# Email accounts
GET   /api/email-accounts
POST  /api/email-accounts/connect/gmail  # returns OAuth URL
GET   /api/email-accounts/oauth/callback/gmail
POST  /api/email-accounts/smtp           # body: host/port/user/pass
DELETE /api/email-accounts/:id
POST  /api/email-accounts/:id/test       # send test mail

# Contacts
GET   /api/contacts?listId&search&page
POST  /api/contacts/import               # multipart CSV
POST  /api/contacts                      # single
PATCH /api/contacts/:id
DELETE /api/contacts/:id
GET   /api/contacts/export.csv

# Lists
CRUD  /api/lists

# Templates
```

```

CRUD    /api/templates

# Campaigns
CRUD    /api/campaigns
POST    /api/campaigns/:id/launch
POST    /api/campaigns/:id/pause
GET     /api/campaigns/:id/stats
GET     /api/campaigns/:id/recipients?status

# Inbox
GET     /api/threads?accountId&status&q
GET     /api/threads/:id
POST    /api/threads/:id/reply           # body uses provider chain
POST    /api/messages/:id/ai-draft      # regenerate AI suggestion

# Workflows
CRUD    /api/workflows
POST    /api/workflows/:id/enable
GET     /api/workflows/:id/runs

# Rewards
CRUD    /api/rewards
GET     /api/rewards/:id/grants

# Analytics
GET     /api/analytics/overview
GET     /api/analytics/campaign/:id
GET     /api/analytics/accounts

# Webhooks (no auth, signed)
POST    /api/webhooks/gmail             # Pub/Sub push
POST    /api/webhooks/sendgrid          # events
POST    /api/webhooks/mailgun
GET     /api/webhooks/track/open/:rid.png # open pixel
GET     /api/webhooks/track/click/:rid   # click redirect
GET     /api/unsubscribe/:token         # public

# Public API for integrations (api key auth)
GET/POST /api/v1/*
```

7. Queue & Worker Design (BullMQ)

Queues

Queue	Concurrency	Purpose
send-email	per-account rate-limited (token bucket in Redis)	dispatch a single email
campaign-fanout	5	expand a campaign into N send-email jobs

Queue	Concurrency	Purpose
inbound-fetch	10	pull messages from Gmail history / IMAP
inbound-process	20	parse, dedupe, store, link to thread/campaign
ai-analyze	10	classify intent + draft reply via OpenRouter
workflow-run	20	execute workflow steps
reward-grant	5	call external webhook/api to grant lifetime premium
account-health	repeat every 5m	refresh Gmail watch, IMAP poll, daily-cap reset
dead-letter	manual	jobs that exhausted retries

Job lifecycle (outbound)

User launches campaign

- campaign-fanout job
- for each contact: create CampaignRecipient(status=queued)
- enqueue send-email{ recipientId, scheduledFor, accountIdHint }

send-email worker:

1. acquire token from SenderRotator (round-robin/weighted/LRU)
2. check token bucket for chosen EmailAccount (Redis)
3. render template with merge tags
4. inject tracking pixel + click-wrap links
5. send via provider adapter (Gmail API / SMTP / SendGrid)
6. save Message(direction='out'), update CampaignRecipient.status='sent'
7. on failure: exponential backoff, attempt<5; else DLQ + AuditLog

Job lifecycle (inbound)

```
Gmail Pub/Sub push → /api/webhooks/gmail
  → enqueue inbound-fetch{ accountId, historyId }
inbound-fetch:
  → call gmail.users.history.list since stored historyId
  → for each new message: enqueue inbound-process{ accountId, gmailMsgId }
inbound-process:
  → fetch full message, parse with mailparser
  → find or create Thread (by References/In-Reply-To, then subject)
  → link to Contact + CampaignRecipient when possible
  → save Message(direction='in')
  → enqueue ai-analyze{ messageId }
ai-analyze:
  → OpenRouter call → intent, confidence, entities, draftReply
  → update Message.ai + Thread.aiIntent/aiSummary
  → emit event: 'reply_received' → workflow engine
workflow-run:
  → match enabled workflows where trigger fits
  → evaluate conditions, execute actions
  → action 'grant_reward' → enqueue reward-grant
```

Rate limiting

Token bucket per `emailAccountId` stored in Redis (Lua atomic script). Limits configurable per account (e.g., Gmail 500/day, custom SMTP higher). Buckets refill on `account-health` schedule.

8. Auth.js v5 Flow

- Providers: Credentials (email+password, bcrypt) + Google OAuth (used both for login AND as the consent flow for connecting a Gmail account — second OAuth scope set `gmail.send`, `gmail.readonly`, `gmail.modify`).
- Session: JWT in cookie (stateless) → middleware reads, attaches `{userId, orgId, role}` to request.
- RBAC: simple `requireRole(req, ['admin'])` helper in `apps/web/lib/auth.ts`.
- Multi-tenancy: every API handler resolves `orgId` from session; Mongoose queries always include `{orgId}`. Add a tiny `withOrg` wrapper to enforce.
- Org switching: a user can belong to many orgs via `Membership` link doc (optional v2).

9. AI Automation Flow (OpenRouter)

Single env var `OPENROUTER_API_KEY`. Wrapper picks model per task:

Task	Model (default)	Fallback
Intent classification	anthropic/claude-haiku-4-5	openai/gpt-4o-mini

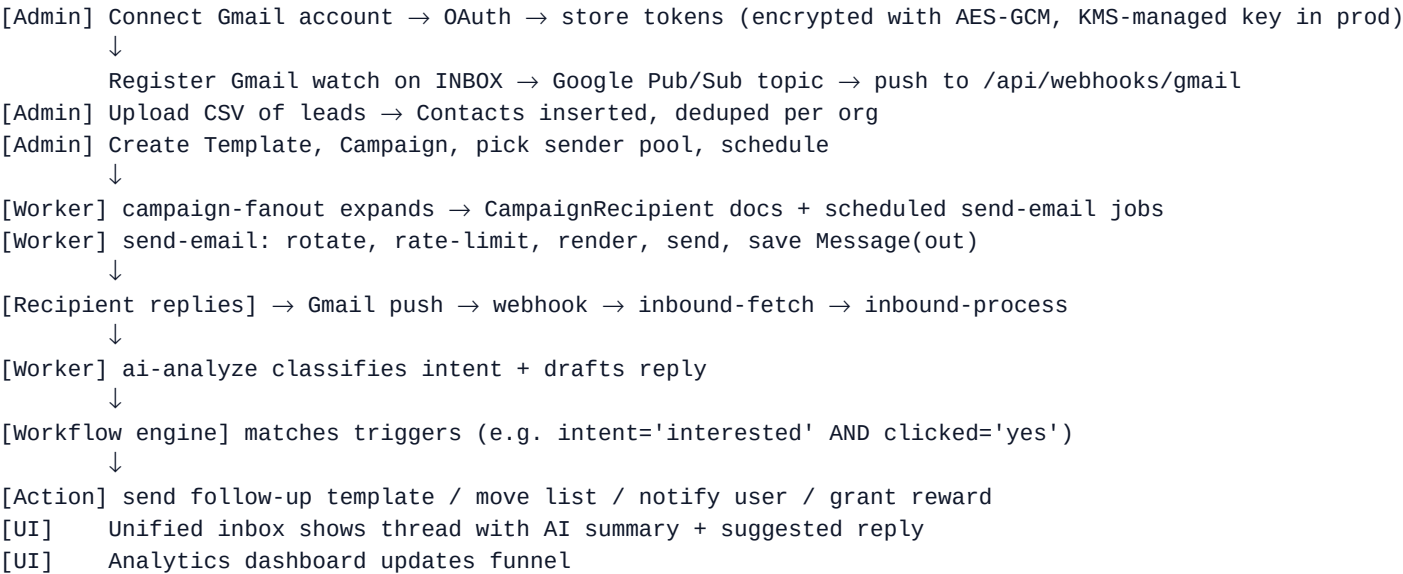
Task	Model (default)	Fallback
Reply draft	anthropic/claude-sonnet-4-6	openai/gpt-4o
Thread summary	anthropic/claude-haiku-4-5	google/gemini-2.0-flash
Embeddings (for similar-thread lookup, v2)	openai/text-embedding-3-small	—

Prompt templates live in `packages/ai/prompts/`, version-controlled. Each AI call logs `{model, tokensIn, tokensOut, costUsd, latencyMs}` for cost dashboard.

Reply analyzer prompt (shape)

- Input: subject, last 5 messages in thread, contact metadata
- Output (JSON, validated by Zod): `{intent, confidence, entities:{...}, summary, suggestedReply}`

10. Email Processing Pipeline (end to end)



11. Dashboard UI Structure

shadcn/ui + Tailwind. App Router segments under `(dashboard)`.

- `/inbox` — three-pane (accounts/folders → thread list → message + AI panel on right)
- `/campaigns` — list table; `/campaigns/new` wizard (5 steps); `/campaigns/:id` overview + recipients tab + stats tab

- `/contacts` — virtualized table (TanStack Table), filters, bulk actions, import drawer
- `/workflows` — list + visual builder (React Flow): trigger → conditions → actions
- `/accounts` — connected mailbox grid with health badges, daily-cap meter, "Send test" action, DNS guidance drawer
- `/analytics` — overview KPIs, per-campaign funnel charts (Recharts), sender health heatmap
- `/settings` — org, members, API keys, webhooks, billing (v2), audit log

Shared chrome: top bar with org switcher, command palette (`cmdk`), notification bell streaming from Redis pub/sub via SSE.

12. Recommended Packages

```
# Core
next, react, typescript, tailwindcss, @radix-ui/*, shadcn/ui (generated), lucide-react

# Auth
next-auth@5 (Auth.js v5), @auth/mongodb-adapter, bcryptjs

# DB
mongoose, mongodb

# Queue
bullmq, ioredis, @bull-board/express, @bull-board/api

# Email
googleapis (Gmail API), nodemailer, mailparser, @sendgrid/mail, mailgun.js
html-to-text, dompurify, isomorphic-dompurify

# AI
openai (compatible client pointed at OpenRouter base URL)
# or `ai` SDK (Vercel) with OpenRouter provider

# Validation / utils
zod, dayjs, uuid, papaparse (CSV), p-queue, pino, pino-pretty

# Workflow UI
reactflow

# Tables / charts
@tanstack/react-table, @tanstack/react-query, recharts

# Observability
@sentry/nextjs, pino

# Tooling
turbo (Turborepo), tsx, eslint, prettier, vitest, playwright
```

13. Step-by-Step Implementation Roadmap

Phase 0 — Bootstrap (½ day)

1. `pnpm create turbo` skeleton; init `apps/web`, `apps/worker`, `packages/*`.
2. `docker-compose.yml` for Mongo + Redis local.
3. `.env.example` with all keys; commit `var lock`-style schema.
4. Tailwind + shadcn/ui init in `apps/web`; base layout, theme toggle.

Phase 1 — Auth & Org (1 day)

5. Auth.js v5 Credentials + Google provider; Mongoose adapter.
6. `User`, `Org`, `Membership` models; signup creates org.
7. Middleware: gated (`dashboard`) routes; `withOrg(handler)` wrapper for API.

Phase 2 — Connect first email account (1 day)

8. Gmail OAuth connect flow; encrypted token storage.
9. `/accounts` page: list, connect, delete, test-send.
10. SMTP connect form.

Phase 3 — Contacts & Templates (1 day)

11. `Contact` + `List` CRUD, CSV import with PapaParse (column mapper), unsubscribe link generator.
12. `Template` CRUD with Handlebars merge-tag preview.

Phase 4 — Send a campaign (MVP-killer feature, 2 days)

13. BullMQ setup in `apps/worker`; queues + connection.
14. `campaign-fanout` + `send-email` workers with token-bucket rate limiter.
15. Campaign wizard UI; launch button enqueues fanout.
16. Tracking pixel route + click-wrap redirect route; record events.

Phase 5 — Inbound + unified inbox (2 days)

17. Gmail `users.watch` setup + Pub/Sub topic; webhook handler.
18. `inbound-fetch` + `inbound-process` workers; thread reconciliation.
19. `/inbox` three-pane UI; reply composer using same send pipeline.

Phase 6 — AI layer (1 day)

20. `packages/ai` with OpenRouter client + Zod-validated outputs.

21. `ai-analyze` worker on every inbound message; populate `Message.ai` + `Thread.aiIntent`.
22. Inbox right-panel shows summary + suggested reply + "Send AI reply" button.

Phase 7 — Workflow engine (2 days)

23. Workflow DSL + engine in `packages/workflows`.
24. React Flow visual builder under `/workflows`.
25. Event bus (Redis pub/sub) emits `reply_received`, `no_reply_after`, `link_clicked`; engine subscribes.
26. Action set: send template, change list, tag, notify, grant reward, pause campaign.

Phase 8 — Rewards / lifetime premium (½ day)

27. `Reward` model + grant action (webhook with HMAC signature) wired into workflow.
28. `/rewards` admin UI + grants log.

Phase 9 — Analytics (1 day)

29. Aggregation pipelines for funnel + per-sender health.
30. `/analytics` dashboards with Recharts.

Phase 10 — Polish & launch (1–2 days)

31. Audit logs, API keys, public `/api/v1`, BullBoard at `/admin/queues` (admin only).
32. Sentry + Pino in both apps; healthcheck endpoints.
33. SPF/DKIM/DMARC guidance UI per domain.
34. Playwright happy-path e2e + Vitest unit tests on workers/engine.
35. Production deploy.

Total MVP estimate: ~12 working days for one focused engineer; meaningfully shippable after Phase 6.

14. Production Deployment Strategy

- **Vercel** hosts `apps/web` (Next.js). Edge for marketing, Node runtime for API routes that touch Mongo/Redis. Set `OPENROUTER_API_KEY`, `MONGODB_URI`, `REDIS_URL`, `AUTH_SECRET`, `GOOGLE_CLIENT_ID/SECRET`, `ENCRYPTION_KEY`.
- **Railway** (or Render/Fly) hosts `apps/worker` as a long-running Node service (BullMQ workers + schedulers). Scale horizontally; each replica picks up jobs from the same Redis.
- **MongoDB Atlas** — M10+ for prod; enable backups + alerts.

- **Redis** — Upstash for serverless web API; Railway-hosted Redis for workers (single source of truth, both web + workers connect to it).
- **Gmail Pub/Sub** — GCP project with topic + push subscription pointing at `https://<domain>/api/webhooks/gmail` (verified token in header).
- **DNS** — onboard customers via subdomain `mail.theirdomain.com` for tracking links; show DKIM/SPF setup wizard.
- **Secrets** — Vercel/Railway env vars; rotate `ENCRYPTION_KEY` via re-encrypt job.
- **CI** — GitHub Actions: lint → typecheck → test → build → deploy on main.
- **Observability** — Sentry for both apps; BullBoard behind admin auth; Atlas + Upstash dashboards.

15. Security Best Practices

- Encrypt all third-party tokens/passwords at rest with AES-256-GCM; key from env, rotated quarterly.
- Auth.js JWT with rotating `AUTH_SECRET`; HttpOnly + SameSite=Lax cookies.
- Per-org isolation enforced by `withOrg` middleware — never trust client-supplied `orgId`.
- Zod-validate every API body; sanitize HTML in templates with DOMPurify before send + before render.
- Sign all outbound webhooks (HMAC-SHA256); verify inbound webhooks (Gmail JWT, SendGrid signature, Mailgun signature).
- Rate-limit public endpoints (unsubscribe, tracking pixel) with Upstash Ratelimit.
- Public API keys hashed (bcrypt) before storage; show only at creation.
- Audit log every privileged action (account connect, campaign launch, reward grant, key creation).
- CSP headers, HSTS, no `dangerouslySetInnerHTML` outside the email preview sandbox iframe.
- Secret scanning + Dependabot on the repo.
- GDPR: per-contact unsubscribe + delete; exportable user data; configurable data retention TTL on `AuditLog` and old `Message` bodies.
- Never commit `.env`; `.env.example` only.

16. Critical Files (when implementation starts)

File	Purpose
<code>packages/db/models/*.ts</code>	Mongoose schemas + indexes
<code>packages/queue/queues.ts</code>	BullMQ queue definitions, shared by web + worker
<code>packages/email/sender.ts</code>	SenderRotator + provider adapter dispatch
<code>packages/ai/client.ts</code>	OpenRouter client + per-task model router

File	Purpose
<code>packages/workflows/engine.ts</code>	Trigger→condition→action evaluator
<code>apps/web/middleware.ts</code>	Auth + org gate
<code>apps/web/lib/auth.ts</code>	Auth.js config + <code>withOrg</code> helper
<code>apps/web/app/api/webhooks/gmail/routes.ts</code>	Pub/Sub push handler
<code>apps/worker/src/workers/send-email.worker.ts</code>	The most-load-bearing worker
<code>apps/worker/src/workers/ai-analyze.worker.ts</code>	AI classification + draft

17. Verification (end to end)

After Phase 6 is complete, run this happy-path locally:

1. `docker compose up -d` (mongo + redis), `pnpm dev` (web), `pnpm --filter worker dev` (worker).
2. Sign up → org auto-created.
3. Connect a Gmail account via OAuth, run "Send test" → receive a test mail.
4. Import 5 contacts via CSV.
5. Create a template `Hello {{firstName}}`.
6. Create a campaign targeting the 5 contacts via the connected account → launch.
7. Verify in BullBoard (`/admin/queues`) that `campaign-fanout` → 5x `send-email` jobs complete.
8. Verify 5 inbound mails arrive at the test recipient inbox.
9. Reply to one as the recipient → Gmail push fires within ~10s → `inbound-process` saves Message → `ai-analyze` populates `intent` + `suggestedReply`.
10. Open `/inbox`: thread is visible with AI summary and a one-click suggested reply.
11. Create a workflow `trigger=reply_received, intent=interested` → `action=grant_reward(webhookUrl=...)`; reply again with positive text; verify webhook fires (use `webhook.site`).
12. Open `/analytics`: counters show sent/replied; per-account health is green.
13. Run `pnpm test` (vitest) and `pnpm test:e2e` (playwright) — all green.

If steps 7, 9, and 11 succeed, the platform's three load-bearing pipelines (outbound queue, inbound + AI, workflow → action) are proven and the MVP is shippable.

18. Deliverable Format

This plan will be exported to a **modern, styled PDF** (Tailwind-themed, dark accent, monospace code blocks, page numbers, TOC) using the `pdf` skill in a follow-up step **after you approve this plan**. The PDF will preserve all sections above and add a cover page + table of contents.